

Standard math functions

When BC is invoked with the `-l` option, the math library is also loaded, making available for use the following six math functions:

- $s(x)$ – returns sine of x (x is radians)
- $c(x)$ – returns cosine of x (x is radians)
- $a(x)$ – returns arctangent (in radians) of x
- $l(x)$ – returns the natural logarithm (base e) of x
- $e(x)$ – returns the value of e raised to the power of x
- $j(n, x)$ – Bessel function of integer order n of x

All these functions operate with the scale dictated by the built-in variable `scale`, which is by default, set to 20. Here's a sample execution:

```
$ bc -ql
scale # Show the current scale
20
pi=4*a(1) # Calculate pi as tan^-1(1) is pi / 4
pi # Show the value approx. to 20 decimals
3.14159265358979323844
s(pi/3) # Calculate sine of 60° - should sqrt(3)/2
.86602540378443864675
sqrt(3)/2 # value for comparison - note the approx. error
.86602540378443864676
c(pi/3) # Calculate sine of 60° - should be 0.5
.50000000000000000001
l(1) # log(1)
0
e(1) # Value of e^1 approx. to 20 decimals
2.71828182845904523536
quit
```

If all this sounds too geeky and mathematical, let's forget it, and do some simple stuff; let's write simple user-defined functions.

User-defined functions

Here is how you write a user-defined function (to add two numbers) in BC:

```
$ bc -ql
define add(x, y) {
    return (x + y)
}
add(3, add(4, 5)) # Lets add 3 with the sum of 4 & 5
12
quit
```

Given that, the factorial code from our previous learnings can be converted into a function as follows (say, in `functions.bc`):

```
define factorial(n) {
    product = 1
    for (current_num = 1; current_num <= n; current_num
+= 1)
```

```
{
    product *= current_num
}
return product
}
```

And then, you can use that function, as follows:

```
$ bc -ql functions.bc # Load the functions while invoking
bc
factorial(10) # Compute the factorial of 10
3628800
quit
```

As you now have `factorial`, you can even calculate the series of e , i.e., $1 + 1/1! + 1/2! + \dots$, say up to $1/20$ for a good enough approximation. Here's how it would go:

```
$ bc -ql functions.bc
exp=1
for (i = 1; i <= 20; i++)
{
    exp += (1/factorial(i))
}
exp # Display the computed value of e
2.71828182845904523525
e(1) # Compare with the standard math function
2.71828182845904523536
quit
```

And, as in C, if you need a function only to do actions and not return anything, `void` is the way:

```
$ bc -ql
define void designer_print(v) {
    print "---(", v, ")---"
}
designer_print(100) # Print 100 with the designs
---(100)---
quit
```

Having covered the fundamentals of these functions in BC, we will dive into its recursive functional power in the next issue. 

By: Anil Kumar Pugalia

The author is an open source hardware and software hobbyist, with a passion for mathematics. His exploration with mathematics, in every aspect of life, dates back to the 1990s. A gold medalist from the Indian Institute of Science, mathematics and knowledge-sharing are two of his many passions. Apart from that, he experiments with Linux and embedded systems to share his learnings through his weekend workshops. Learn more about him and his experiments at <http://sysplay.in>. He can be reached at email@sanka-pugs.com.